

Lecture Notes: Time Complexity of Python Operations

Source: Code & Debug (DSA Python Course 2025 - Part 4)

1. Introduction & Importance

One of the most critical aspects of Data Structures and Algorithms (DSA) is understanding the internal time complexity of built-in operations.

If you use a function like 'pop(0)' inside a loop without knowing it takes $O(N)$ time, your solution may become $O(N^2)$ and fail in an interview.

Goal: Learn the cost of operations for Lists, Sets, and Dictionaries.

2. Python Lists (Dynamic Arrays)

Lists are the most common data structure. Performance depends on whether elements need to be shifted.

Constant Time - $O(1)$

- Append (list.append(x)): Adds to the end. No shifting.
- Pop Last (list.pop()): Removes the last item. No shifting.
- Indexing (list[i]): Direct access to memory address.
- Length (len(list)): Python stores the length attribute; it does not count items iteratively.

Linear Time - $O(N)$

- Pop Intermediate (list.pop(0)): Removing the first item forces ALL subsequent items to shift left.
- Insert (list.insert(0, x)): Inserting at the start forces ALL items to shift right.
- Delete (del list[i]): Requires shifting to fill the gap.
- Iteration (for x in list): Visits every element.
- Membership (x in list): Performs a linear search (scans one by one).

```
# Example of Hidden Complexity
my_list = [1, 2, 3, 4, 5]
if 10 in my_list: # This is  $O(N)$ 
    print('Found')
my_list.pop(0)    # This is  $O(N)$  due to shifting
```

3. Python Sets (Hash Tables)

Sets use Hashing. This allows for extremely fast lookups compared to Lists.

Note: Sets do not have order or indexing.

Membership (x in set):

- Average Case: $O(1)$ (Instant lookup via hash).
- Worst Case: $O(N)$ (Occurs rarely due to hash collisions).

Lecture Notes: Time Complexity of Python Operations

Source: Code & Debug (DSA Python Course 2025 - Part 4)

```
my_set = {1, 2, 3}
if 3 in my_set:      # This is O(1) - Much faster than List!
```

4. Python Dictionaries (Hash Maps)

Dictionaries also rely on Hashing (Keys are hashed). Operations are generally $O(1)$.

- Get Item (`d[key]`): Avg $O(1)$.
- Set Item (`d[key] = val`): Avg $O(1)$.
- Delete Item (`del d[key]`): Avg $O(1)$.
- Iteration: $O(N)$ (Must visit every key-value pair).

5. Summary Table

Operation	List Complexity	Set/Dict Complexity (Avg)

Add to End	$O(1)$	$O(1)$
Pop from End	$O(1)$	$O(1)$
Pop from Start	$O(N)$ (Shift)	$O(1)$
Search (in)	$O(N)$	$O(1)$ (Major Advantage)
Get Length	$O(1)$	$O(1)$